Lectura y analisis de señales ECG

Kevin Javier Gonzalez Luna, Iván Felipe Maluche Suarez, Angélica Pascagaza Vega
Universidad Nacional de Colombia
Sede Bogotá
Bogotá D.C., Colombia
{kegonzalezl, imaluche, apascagaza}@unal.edu.co

***Resumen*—Este documento presenta un estudio y posterior implementacion de un lector de señales ECG por medio de las estructuras de datos aprendidas a lo largo del curso, para esto el grupo se basa en diferentes papers y estudios que permitan comprender a fondo la lectura de estas señales y como mostrarlas y leerlas adecuadamente**

***Index Terms*—Medicina, Electro cardiograma, estructuras de datos, algoritmos, instrumentos de medicion.**

I. Introducción

En el campo de la cardiologia debido a la complejidad y gravedad de las enfermades se hace de especial importancia el uso de herramientas capaces de leer y analizar datos clinicos de los pacientes de manera que se pueda advertir de posibles riesgos para su salud, sabiendo como esta tarea puede encontrar solucion en la programacion y las estructuras de datos como grupo vemos de especial interes hacer uso de una implementacion de lo visto en el curso para clasificar los datos clinicos de un paciente de forma que se puedan leer y analizar de manera eficiente por parte del personal de salud.

Para este fin en nuestro proyecto se estudiara la lectura, analisis y graficacion de señales ECG (obtenidas a base de un electrocardiograma) para ser capaces de entregar un reporte en el que se puedan identificar patrones anomalos.

I-A. Definicion del proyecto

La idea central del proyecto consiste en por medio de las implementaciones estudiadas en el curso de estructuras de datos manejar los datos del ECG y detectar tanto propiedades como anomalias de este, esto puede permitir inovacioens y mejoras en cuanto a eficiencia, algo necesario para facilitar el trabajo para el personal medico.

I-B. Objetivos

- **Ser capaz de leer registros de la base de datos MIT-BIH Arrhythmia Database incluyendo los archivos de registro y las anotaciones del cardiologo.** [2]
- **Hacer un uso activo de estructuras de datos capaces de permitir un recibiendo y envio eficiente de datos para el procesamiento y filtrado de informacion**
- **Entregar un resultado facil de leer para el cardiologo de manera que permita analizar los datos facilmente**

II. requerimientos basicos de la implementacion

Esta seccion se enfoca en los requerimientos mas basicos que debe cumplir el codigo.
Para cumplir con la idea planteada el proyecto debe ser capaz de:

- Leer cada uno de los archivos del conjunto de datos MIT (.head, .dat, .atr y .xws) [2]
- Organizar los datos de manera que puedan ser facilmente leidos y manejados por las estructuras de datos
- Graficar los datos de manera que sean entendibles
- Filtrar ruido para obtener mejores resultados en las propiedades de la señal.

III. Diseño inicial de la solucion

III-A. Orden de los archivos del proyecto

Dentro de la carpeta src disponible junto con este documento se encuentra:

- **graficador:** Contiene los archivos necesarios para graficar la señal ECG y obtener sus datos, no se ahondara directamente en su implementacion al no ser el objeto principal para las implemenaciones de estructuras de datos.
- **implementaciones:** Contendra las implementaciones de estructuras de datos y ADT genericas que se usaran para el proyecto, actualmente se encuentran una implementacion de Cqueues y CLinked List.
- **DeteccionQRS.java:** Contiene el codigo que hace uso de las implementaciones y el graficador para filtrar y mantener un registro de los datos del ECG.

III-B. Loop basico de la implementacion

Por medio del lo archivos dispuestos en la carpeta "graficador"se crea un objeto capaz de graficar correctamente el ECG y entregar sus datos, teniendo esto y las respectivas implementaciones de Linked list y circular queue basandose en el algoritmo de pan-topkins el archivo DeteccionQRS.java se encarga de filtrar los datos por medio de la cqueue y mantener un registro de estos en la linked list. Hecho esto se realizan los calculos y se entregan las propiedades en termianl mientras que la grafica se mantiene en una ventana emergente. [1]

III-C. Futuros aspectos a implementar

Tomando en cuenta lo alcanzado en esta primera etapa esperamos poder a futuro implementar mas estructuras para la toma de decisiones y analisis de propiedades de la señal, asi como mejorar el formato de salida principalmente para las propiedades

IV. Estructuras utilizadas

Para las implementaciones de la primera fase del proyecto se han utilizado circular queues y Circular linked lists.

IV-A. motivos de seleccion

- Circular queue: Se decidio usar para el filtrado de señal al saber exactamente la cantidad de datos que se almacenaria y al tener un costo de tan solo O(1) para ingreso y salida de datos, ademas de que las FIFOS ecajaban adecuadamente con el proceso de filtrado
- Circular linked list: Se uso para mantener un registro al ser datos de los cuales no se conoce la cantidad exacta de antemano, permitiendo un uso de memoria menos costoso

IV-B. Estructuras pensadas para futuras fases

Por el momento la principal estructura pensada para una futura fase son los BST, estos se podria usar para por medio de umbrales de decision ser capaces de clasificar los latidos leidos del ECG.

V. Primera implementacion

Como se ha mencionado a lo largo del documento se implemento un algoritmo basado en pan-Tompkins para el filtrado de señales ECG, su implementacion se realizo de la siguiente manera:

Pseudocódigo del Algoritmo Pan-Tompkins

V-A. 1. Etapas de Filtrado y Procesamiento

En esta primera etapa se desarrollaron las funciones de filtrado de datos y procesamiento de muestras para despues ser aplicadas con la ADT

Algorithm 1 Procesamiento de Muestra

```
function ProcesarMuestra(m)
    if not tieneOffset then
        offset ← m
        tieneOffset ← true
    end if
    m ← m − offset
    pb ← PasaBajos(m)
    pa ← PasaAltos(pb)
    derivar ← Derivada(pa)
    elevarCuadrado ← Cuadrado(derivar)
    ventana ← IntegracionVentana(elevarCuadrado)
    return ventana
end function
```

Algorithm 2 Filtro Pasa-Bajos

```
function PasaBajos(x)
    Desplazar y almacenar x en buffer Pasa-Bajos
    y ← 2·y[n−1]−y[n−2]+x[n]−2·x[n−6]+x[n−12]
    Almacenar y en buffer de salidas Pasa-Bajos
    return y
end function
```

Algorithm 3 Filtro Pasa-Altos

```
function PasaAltos(x)
    Desplazar y almacenar x en buffer Pasa-Altos
    y ← y[n−1] − x[n]/32.0 + x[n−16] − x[n−15] + x[n−32]/32.0
    Almacenar y en buffer de salidas Pasa-Altos
    return y
end function
```

Algorithm 4 Derivada

```
function Derivada(x)
    Desplazar y almacenar x en buffer de Derivada
    y ← 0.125·(2·x[n]+x[n−1]−x[n−3]−2·x[n−4])
    return y
end function
```

Algorithm 5 Cuadrado e Integración de Ventana

```
function Cuadrado(x)
    return x · x
end function
function IntegracionVentana(x)
    datoAntiguo ← Sacar el dato más antiguo del buffer de integración
    suma_int ← suma_int + x - datoAntiguo
    Desplazar y almacenar x en buffer de integración
    return suma_int / window_size
end function
```

V-B. 2. Fase de Decisión y Lógica de Umbrales

En esta parte del algoritmo a partir de los umbrales se decide el tipo de entrada y se clasifica, actualizando los umbrales a partir de la informacion obtenida por este proceso.

Algorithm 6 Fase de Decisión

```
procedure DECIDIR(v)
    pendMax ← máx(pendMax, v − prev1)
    if not inicializado then
        maxInicializacion ← máx(maxInicializacion, v)
        sumaInicializacion ← sumaInicializacion + v
        if n + 1 ≥ muestrasInicializacion then
            spki ← 0.25 · maxInicializacion
            npki ← 0.5 · (sumaInicializacion/muestrasInicializacion)
            ACTUALIZARUMBRALES
            pendMax ← 0
            inicializado ← true
        end if
    else
        if prev1 > prev2 and prev1 ≥ v then
            CLASIFICAR(prev1, n − 1)
            pendMax ← 0
        end if
        SEARCHBACK
    end if
    prev2 ← prev1
    prev1 ← v
    n ← n + 1
end procedure
procedure ACTUALIZARUMBRALES
    thresholdI ← npki + 0.25 · (spki − npki)
    thresholdII ← 0.5 · thresholdI
end procedure
```

Algorithm 7 Clasificación de Picos y Searchback

```
procedure CLASIFICAR(pico, idx)
    periodoRefactor ← round(0.200 · fs)
    ventanaT ← round(0.360 · fs)
    desde ← idx - ultimoQRS
    if ultimoQRS ≥ 0 and desde < periodoRefactor then
        return
    end if
    if pico > thresholdI then
        ondaT ← (ultimoQRS ≥ 0) and (desde < ventanaT) and (pendMax < 0.5 · pendQRS)
        if ondaT then
            npki ← 0.125 · pico + 0.875 · npki
        else
            spki ← 0.125 · pico + 0.875 · spki
            REGISTRARQRS(idx, pendMax)
        end if
    else
        npki ← 0.125 · pico + 0.875 · npki
        if pico > thresholdII and pico > candPico then
            candPico ← pico
            candIdx ← idx
            candPend ← pendMax
        end if
    end if
    ACTUALIZARUMBRALES
end procedure
procedure SEARCHBACK
    if candIdx ≥ 0 and haySuficientesIntervalosRR and (n − ultimoQRS) > 1.66 · rrReferencia() then
        spki ← 0.25 · candPico + 0.75 · spki
        REGISTRARQRS(candIdx, candPend)
        ACTUALIZARUMBRALES
    end if
end procedure
```

V-C. 3. Clasificación de Picos y Searchback

Este ultimo bloque del algoritmo se encarga de clasificar el tipo de pico a base de su pentiente y nuevamente actualizar los umbrales a partir de la informacion obtenida

Referencias

[1] School of Electrical and Computer Engineering Cornell David A. Duffield College of Engineering. Accedido el 14 de octubre de 2026. [En línea]. Disponible: https://people.ece.cornell.edu/land/courses/ece5030/labs/s2014/Pan_Tompkins.pdf

[2] G. Moody and R. Mark, "MIT-BIH Arrhythmia Database,"PhysioNet, version 1.0.0, feb. 2005. doi: 10.13026/C2F305